1. Two Sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Here are **multiple Java solutions** for the **Two Sum** problem — from brute force to the optimal approach.
Each version includes an explanation to help you understand the logic clearly.

---

## ✅ 1. Brute Force Solution ($O(n^2)$)

Check every pair.

### Code

java Copy code

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{}; // never happens because solution guaranteed
    }
}
```

### ✔ Pros

- Easiest to understand

- No extra data structure

### ❌ Cons

- Slow when array is large

---

## ✅ 2. HashMap Solution (Optimal O(n))

Use a HashMap to store value → index.

For each element, check if `(target - nums[i])` already exists in map.

### Code

java Copy code

```java
import java.util.HashMap;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];

            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }

            map.put(nums[i], i);
        }

        return new int[]{}; // guaranteed solution exists
    }
}
```

### ✔️ Pros

- Fast
- Only one pass
- Most commonly used in interviews

---

## ✅ 3. Two-Pointer Solution (O(n log n)) — Only Works if Sorted

⚠️ But **this cannot be used** for this exact problem because sorting changes the indices. Still, it's useful to know for variants.

### Code (illustration only)

java Copy code

```java
import java.util.Arrays;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        int[][] arr = new int[nums.length][2];
        for (int i = 0; i < nums.length; i++) {
            arr[i][0] = nums[i];
            arr[i][1] = i;
        }

        Arrays.sort(arr, (a, b) -> a[0] - b[0]);

        int left = 0, right = nums.length - 1;

        while (left < right) {
            int sum = arr[left][0] + arr[right][0];

            if (sum == target) {
                return new int[]{arr[left][1], arr[right][1]};
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
        return new int[]{};
    }
}
```

---

## ⭐ Recommended Approach: HashMap (O(n))

Interviewers expect this solution.

If you want, I can also give you:

- ✔ Dry run examples
- ✔ Diagram explanation
- ✔ Step-by-step tracing
- ✔ Practice variations (3Sum, 4Sum, Two Sum II, etc.)

Binary Tree Inorder Traversal

Given the root of a binary tree, return the inorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2

Below are **three ways** to solve *Binary Tree Inorder Traversal* in Java:

- ✔ Recursive
- ✔ Iterative using Stack
- ✔ Morris Traversal (O(1) space)

I'll also explain the example.

---

## ✅ What Is Inorder Traversal?

**Left → Node → Right**

---

## ✔ Example

Input:

```
[1, null, 2, 3]
```

This tree looks like:

markdown Copy code

```
    1
     \
      2
     /
    3
```

**Inorder traversal:**

Left of 1 → 1 → Left of 2 → 3 → 2

**Output = [1, 3, 2]**

---

## ✅ 1. Recursive Solution (Simplest)

Time: **O(n)**

Space: **O(n)** due to recursion stack

java Copy code

```java
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        inorder(root, result);
        return result;
    }

    private void inorder(TreeNode node, List<Integer> result) {
        if (node == null) return;

        inorder(node.left, result);   // left
        result.add(node.val);         // root
        inorder(node.right, result);  // right
    }
}
```

---

## ✅ 2. Iterative Solution Using Stack (No Recursion)

Time: **O(n)**

Space: **O(n)**

java Copy code

```java
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        Stack<TreeNode> stack = new Stack<>();
        TreeNode curr = root;

        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;   // keep going left
            }

            curr = stack.pop();      // process node
            result.add(curr.val);

            curr = curr.right;       // go to right
        }

        return result;
    }
}
```

---

## ✅ 3. Morris Traversal (O(1) Space, Advanced)

Time: **O(n)**

Space: **O(1)**

No recursion, no stack.

java Copy code

```java
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        TreeNode curr = root;

        while (curr != null) {
            if (curr.left == null) {
```

```
                result.add(curr.val);
                curr = curr.right;
            } else {
                TreeNode prev = curr.left;
                while (prev.right != null && prev.right != curr) {
                    prev = prev.right;
                }

                if (prev.right == null) {
                    prev.right = curr;   // create thread
                    curr = curr.left;
                } else {
                    prev.right = null;   // break thread
                    result.add(curr.val);
                    curr = curr.right;
                }
            }
        }
        return result;
    }
}
```

---

## ⭐ Recommended for Interviews

Use **Iterative with Stack** OR **Recursive**.

---

If you want, I can also provide:

📌 Preorder & Postorder solutions

📌 All 3 traversals in one function

📌 A step-by-step dry run for the example

how to explain to another person in english but my engilsh are not good can create dialgoles formates

Sure! Here are **simple, easy English dialogue formats** you can use to explain a coding problem (like Binary Tree Inorder Traversal) to another person.
I'll make the English very simple so you can speak confidently.

---

## ✅ Dialogue Format 1: Simple Explanation

**You:**
"Let me explain Inorder Traversal.
Inorder means we visit the tree in three steps:
first Left, then Node, then Right."

**Other Person:**
"Okay, what does that mean?"

**You:**
"We first go to the left child of the node.
When there is no more left child, we print the value.
Then we go to the right side.
So the order is: left → root → right."

---

## ✅ Dialogue Format 2: Explaining Example Tree

**You:**
"Let's take this example: `[1, null, 2, 3]`.
This is the tree:"

markdown Copy code

```
1
 \
  2
 /
3
```

**Other Person:**
"How do we do inorder here?"

**You:**
"First we check the left of 1.

It is empty, so we print 1.
Then go to 2.
Before printing 2, we go to the left, which is 3.
So we print 3, then 2."

**Other Person:**
"So the answer is [1, 3, 2]?"

**You:**
"Yes!"

---

## ✅ Dialogue Format 3: Explaining the Code (Simple English)

**Other Person:**
"How does your code work?"

**You:**
"In the code, I use a stack.
I keep pushing left nodes into the stack.
When no more left nodes, I pop from stack and print the value.
Then I move to the right child.
This repeats until stack is empty."

**Other Person:**
"Oh, so stack helps in storing nodes?"

**You:**
"Yes. Stack helps us remember the path.
This way we can do inorder without recursion."

---

## ✅ Dialogue Format 4: For Very Basic English

**You:**
"Inorder means three steps.
Step 1: Go left.
Step 2: Print number.
Step 3: Go right."

**Other Person:**

"Why like that?"

**You:**

"Because that is the rule.

This order gives sorted output for a Binary Search Tree."

---

## If you want, I can also create:

- ✔ Dialogue for any DSA topic
- ✔ Dialogue for interview explanation
- ✔ Dialogue to explain Java code
- ✔ Very simple English speaking practice

Just tell me the topic!

Dialogue for interview explanation

Sure! Here are **ready-made interview dialogue scripts** you can use in simple, clear English.

This will help you confidently explain a DSA problem or Java concept in an interview.

---

## ⭐ 1. Dialogue — Explaining a DSA Problem (Two Sum Example)

**Interviewer:**

"Can you explain your solution for Two Sum?"

**You:**

"Yes sir. This problem wants two numbers that add to the target.

My approach uses a HashMap because it gives fast lookup."

**Interviewer:**

"How does your method work?"

**You:**

"I go through the array one time.

For each number, I calculate the complement: target – number.

If the complement is already in the map, I have the answer.

If not, I store the current number in the map."

**Interviewer:**

"Why is this better?"

**You:**

"It is better because it works in O(n) time.

Brute force takes $O(n^2)$.

So HashMap is much faster."

---

## ⭐ 2. Dialogue — Explaining Binary Tree Inorder Traversal

**Interviewer:**

"Explain inorder traversal."

**You:**

"Inorder means we visit nodes in this order: Left → Node → Right.

So first we go to the left child, then visit the node, and then go to the right child."

**Interviewer:**

"How do you implement it?"

**You:**

"There are two common ways.

One is recursion.

Second is using a stack."

**Interviewer:**

"Which one did you choose and why?"

**You:**

"I used the stack method.

Because it avoids recursion, and it is easy to understand.

The idea is:

• go left and push all nodes in the stack

• when no more left, pop from stack and add value
• then go right"

**Interviewer:**
"Good. What is time complexity?"

**You:**
"O(n), because we visit each node exactly once."

---

## ⭐ 3. Dialogue — Explaining Java OOP (Simple)

**Interviewer:**
"What is OOP?"

**You:**
"OOP means Object-Oriented Programming.
It organizes code using objects instead of functions.
Java uses four main principles:
• Encapsulation
• Inheritance
• Polymorphism
• Abstraction"

**Interviewer:**
"Can you explain encapsulation?"

**You:**
"Encapsulation means hiding data and giving controlled access.
We use private variables and public getters and setters."

---

## ⭐ 4. Dialogue — Explaining a Java Project

**Interviewer:**
"Tell me about your project."

**You:**
"My project is a decentralized renewable energy marketplace.
I used blockchain for transparency and smart contracts for automation.

Users can buy and sell renewable energy directly.
I also used demand-side management to balance energy usage."

**Interviewer:**
"What was your role?"

**You:**
"My role was to build the backend logic and smart contract integration.
I designed the architecture and implemented core features."

---

## ⭐ 5. Dialogue — Explaining Time Complexity

**Interviewer:**
"Explain time complexity in simple words."

**You:**
"Time complexity tells how fast our algorithm grows when input size grows.
For example, O(n) means time grows linearly.
$O(n^2)$ means time grows much faster.
We use it to compare algorithms."

---

## ⭐ Want dialogues for HR round also?

Or topic-specific dialogues (Java, DBMS, OS, DSA, Projects)?

Same Tree

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Below is the **solution + explanation + interview-style dialogue** for the **Same Tree** problem.

---

## ✅ Same Tree – Simple Logic

Two trees are the same if:

1. Both nodes are **null** → they match
2. One is null and the other is not → **not same**
3. Values are different → **not same**
4. Values are same → check **left subtree** and **right subtree**

---

## ✅ 1. Recursive Java Solution (Best + Easiest)

java Copy code

```java
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        // Case 1: both are null
        if (p == null && q == null) return true;

        // Case 2: one is null or values differ
        if (p == null || q == null || p.val != q.val) return false;

        // Case 3: values are same, check left & right subtrees
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

Time Complexity: **O(n)**
Space: **O(h)** (h = height of tree)

---

## ✅ 2. Iterative Solution Using Queue (Breadth-First Search)

(Useful if interviewer asks for non-recursive version)

java Copy code

```java
import java.util.*;

class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(p);
        queue.add(q);

        while (!queue.isEmpty()) {
            TreeNode a = queue.poll();
            TreeNode b = queue.poll();

            if (a == null && b == null) continue;
            if (a == null || b == null) return false;
            if (a.val != b.val) return false;

            queue.add(a.left);
            queue.add(b.left);
            queue.add(a.right);
            queue.add(b.right);
        }

        return true;
    }
}
```

---

## ⭐ Interview Dialogue (Easy English)

**Interviewer:**

"Can you explain how you check if two trees are the same?"

**You:**

"Yes sir.

Two trees are same if both structure and values are same."

**Interviewer:**

"How do you check that?"

**You:**

"I use recursion.

If both nodes are null, they match.
If one is null and the other is not, they don't match.
If values are different, they don't match.
If values are same, I compare the left subtrees and the right subtrees."

**Interviewer:**
"What is the time complexity?"

**You:**
"O(n), because I visit each node once."

**Interviewer:**
"Why recursion works well here?"

**You:**
"Because tree problems naturally fit recursive structure —
each tree is made of subtrees.
So checking left and right recursively is simple and clean."

---

If you'd like, I can also provide:

- ✔ Dry run example
- ✔ Visual diagram
- ✔ More interview dialogues
- ✔ Similar problems: Symmetric Tree, Subtree of Another Tree, Mirror Tree

Symmetric Tree

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Here is a **clear explanation**, **Java solution**, and an **interview-style dialogue in simple English** for the **Symmetric Tree** problem.

---

# 🌳 Symmetric Tree – Concept

A tree is **symmetric** if:

👉 The **left subtree** is a **mirror** of the **right subtree**.

Mirror means:

- left child of left subtree == right child of right subtree
- right child of left subtree == left child of right subtree

---

## ✅ 1. Recursive Java Solution (Best + Simplest)

java Copy code

```java
class Solution {
    public boolean isSymmetric(TreeNode root) {
        if (root == null) return true;
        return isMirror(root.left, root.right);
    }

    private boolean isMirror(TreeNode t1, TreeNode t2) {
        if (t1 == null && t2 == null) return true;
        if (t1 == null || t2 == null) return false;

        return (t1.val == t2.val)
            && isMirror(t1.left, t2.right)
            && isMirror(t1.right, t2.left);
    }
}
```

✔ Time: **O(n)**
✔ Space: **O(h)** (height of tree)
✔ Most common interview answer

---

## ✅ 2. Iterative Solution Using Queue (BFS)

(Useful if interviewer says: “Can you do without recursion?”)

java Copy code

```java
class Solution {
    public boolean isSymmetric(TreeNode root) {
        if (root == null) return true;

        Queue<TreeNode> q = new LinkedList<>();
        q.add(root.left);
        q.add(root.right);

        while (!q.isEmpty()) {
            TreeNode t1 = q.poll();
            TreeNode t2 = q.poll();

            if (t1 == null && t2 == null) continue;
            if (t1 == null || t2 == null) return false;
            if (t1.val != t2.val) return false;

            q.add(t1.left);
            q.add(t2.right);
            q.add(t1.right);
            q.add(t2.left);
        }

        return true;
    }
}
```

---

## 🌟 Diagram Example

For input:

`[1, 2, 2, 3, 4, 4, 3]`

Tree looks like:

markdown Copy code

```
        1
      /   \
     2     2
    / \   / \
   3  4  4   3
```

This tree **is symmetric**.

---

## 🗣️ Interview Dialogue (Simple English)

**Interviewer:**

"Can you explain your approach for Symmetric Tree?"

**You:**

"Yes sir.

A tree is symmetric if the left subtree is a mirror of the right subtree."

**Interviewer:**

"What do you mean by mirror?"

**You:**

"Mirror means values must match, but in opposite positions.

So left child of one subtree must match the right child of the other subtree."

**Interviewer:**

"How do you check this?"

**You:**

"I use a recursive function `isMirror(t1, t2)`.

First, if both nodes are null, they match.

If one is null, they don't match.

If values are different, not symmetric.

If values are same, I check:

- left of t1 with right of t2
- right of t1 with left of t2"

**Interviewer:**

"Time complexity?"

**You:**

"O(n), because we visit every node once."

---

## Want practice dialogues for more DSA problems?

I can create simple-English dialogues for:

- ✔ Level Order Traversal
- ✔ Max Depth of Binary Tree
- ✔ Path Sum
- ✔ Balanced Binary Tree
- ✔ Insert/Delete in BST

Just tell me the topic!

Merge Two Sorted Lists

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Below is a **simple explanation**, **Java solution**, and an **interview-style dialogue (easy English)** for **Merge Two Sorted Lists**.

---

## ✅ Understanding the Problem

You have two sorted linked lists:

rust Copy code

```
list1: 1 -> 3 -> 5
list2: 2 -> 4 -> 6
```

Your job:
Merge them into one sorted list **without creating new nodes**.

Output:

rust Copy code

```
1 -> 2 -> 3 -> 4 -> 5 -> 6
```

---

## ⭐ 1. Iterative Java Solution (Most Common & Best)

java Copy code

```java
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(-1);
        ListNode current = dummy;

        while (list1 != null && list2 != null) {
            if (list1.val < list2.val) {
                current.next = list1;
                list1 = list1.next;
            } else {
                current.next = list2;
                list2 = list2.next;
            }
            current = current.next;
        }

        // Attach remaining nodes
        if (list1 != null) current.next = list1;
        else current.next = list2;

        return dummy.next;
    }
}
```

- ✔ Time: **O(n + m)**
- ✔ Space: **O(1)**
- ✔ Uses pointers, best for interviews

---

## ⭐ 2. Recursive Java Solution (Clean & Short)

java Copy code

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        if (list1 == null) return list2;
        if (list2 == null) return list1;

        if (list1.val < list2.val) {
            list1.next = mergeTwoLists(list1.next, list2);
            return list1;
        } else {
            list2.next = mergeTwoLists(list1, list2.next);
            return list2;
        }
    }
}
```

- ✔ Looks clean
- ✔ But recursion uses extra stack space

---

## 🧠 Logic Explained in Simple Words

1. Compare the first elements of both lists.
2. Choose the smaller one to add to the merged list.
3. Move that list's pointer forward.
4. Continue until one list becomes empty.
5. Attach whatever is left from the other list.

---

## 🗣️ Interview Dialogue (Very Simple English)

**Interviewer:**

"Explain how you merge two sorted linked lists."

**You:**

"I use two pointers, one for each list.

I compare the values.

Whichever is smaller, I add that node to the answer list and move that pointer ahead."

**Interviewer:**

"What if one list becomes empty?"

**You:**

"Then I attach the remaining part of the other list directly, because it is already sorted."

**Interviewer:**

"Why do you use a dummy node?"

**You:**

"The dummy node makes it easy to build the new list.
I don’t need to handle special cases for the first node."

**Interviewer:**

"Time complexity?"

**You:**

"O(n + m) because I visit every node once."

**Interviewer:**

"Space complexity?"

**You:**

"O(1) for the iterative version."

---

If you want, I can also create dialogue for:

- ✔ Reverse Linked List
- ✔ Remove Duplicates
- ✔ Cycle Detection
- ✔ Add Two Numbers
- ✔ Middle of Linked List

Remove Duplicates from Sorted List

Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Here is a **simple explanation**, **Java solution**, and an **easy-English interview dialogue** for **Remove Duplicates from Sorted List**.

---

## ✅ Understanding the Problem

Linked list is **sorted**, like:

Copy code

```
1 → 1 → 2 → 3 → 3 → 3 → 4
```

We must remove duplicates so that each value appears only once:

Copy code

```
1 → 2 → 3 → 4
```

---

## ⭐ Java Solution (Iterative, Best Approach)

java Copy code

```java
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode current = head;

        while (current != null && current.next != null) {
            if (current.val == current.next.val) {
                current.next = current.next.next; // skip duplicate
            } else {
                current = current.next; // move forward
            }
        }

        return head;
    }
}
```

✔ **Time: O(n)**

✔ **Space: O(1)**

✔ **We don’t create new nodes, only adjust pointers**

---

## 🧠 Logic in Very Simple Words

1. Start from the head.
2. If the current node and the next node have the **same value**, skip the next node.
3. If values are different, move to the next node.
4. Continue until the end of the list.

---

## ⭐ Example Walkthrough

List: `1 → 1 → 2`

- current = 1
- next = 1 → duplicate → skip second 1
  List becomes → `1 → 2`
- move current to 2
  Done.

---

## 🗣️ Interview Dialogue (Easy English)

**Interviewer:**
"Explain how you remove duplicates from a sorted linked list."

**You:**
"Because the list is sorted, all duplicates will be next to each other.
So I just walk through the list and check if the current node and next node have the same value."

**Interviewer:**
"What do you do when values match?"

**You:**
"I skip the next node by linking current.next to current.next.next.
This removes the duplicate."

**Interviewer:**
"And when values don't match?"

**You:**

"Then I move the current pointer forward."

**Interviewer:**

"Time complexity?"

**You:**

"O(n), because I visit each node once."

**Interviewer:**

"Space complexity?"

**You:**

"O(1), because I only use pointers."

---

If you want, I can make **dialogues for Linked List Cycle, Reverse List, Remove Nth Node, Add Two Numbers**, or any other question!

2/2

Linked List Cycle

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.

Here is a **clear explanation**, **Java solution**, and a **simple-English interview dialogue** for **Linked List Cycle**.

---

## 🔥 Understanding the Problem

A **cycle** exists if a node points back to some previous node:

Example:

markdown Copy code

```
1 → 2 → 3 → 4
      ↑     |
      └─────┘
```

Here, after 4, the list points back to 2 → cycle detected.

If no cycle:

csharp Copy code

```
1 → 2 → 3 → null
```

Return `false`.

---

## ⭐ Best Approach: Floyd's Cycle Detection (Tortoise & Hare)

Use two pointers:

- **Slow pointer** moves 1 step
- **Fast pointer** moves 2 steps

If they ever meet → cycle exists.
If fast reaches `null` → no cycle.

---

## ✅ Java Solution (Optimal)

java Copy code

```java
class Solution {
    public boolean hasCycle(ListNode head) {
        if (head == null || head.next == null) return false;

        ListNode slow = head;
```

```
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;           // move 1 step
            fast = fast.next.next;      // move 2 steps

            if (slow == fast) {         // collision → cycle exists
                return true;
            }
        }

        return false; // fast reached null → no cycle
    }
}
```

✔ **Time: O(n)**

✔ **Space: O(1) (no extra memory)**

✔ **Most standard interview solution**

---

## 🧠 Logic in Simple Words

1. Slow moves one step.
2. Fast moves two steps.
3. If there is a cycle, fast will eventually catch slow (like a racing track).
4. If fast reaches `null`, no cycle.

---

## ⭐ Example

List: `1 → 2 → 3 → 4 → 2`

- slow at 1, fast at 1
- slow → 2, fast → 3
- slow → 3, fast → 2
- slow → 4, fast → 4 → they meet → **cycle exists**

---

## 🗣️ Interview Dialogue (Easy English)

**Interviewer:**

"How do you detect a cycle in a linked list?"

**You:**

"I use Floyd's Cycle Detection algorithm.

It uses two pointers: slow and fast.

Slow moves one step, fast moves two steps."

**Interviewer:**

"Why does this work?"

**You:**

"If the list has a cycle, fast will eventually catch the slow pointer inside the loop.

If the list has no cycle, the fast pointer will reach null."

**Interviewer:**

"Time complexity?"

**You:**

"O(n), because both pointers visit nodes in linear time."

**Interviewer:**

"Space complexity?"

**You:**

"O(1), because we only use two pointers."

---

If you want, I can also give:

- ✔ Diagram explanation
- ✔ Cycle starting point solution (Linked List Cycle II)
- ✔ More interview dialogues

Just tell me!

given are problem in how many approaches

For the **Linked List Cycle** problem, there are **3 main approaches** you can use.
Here they are (simple English + interview-friendly):

---

## 🔥 1. Floyd's Cycle Detection (Tortoise & Hare) — Best Approach

✔ Time: **O(n)**
✔ Space: **O(1)**

**Idea:**

- Use a slow pointer (1 step)
- Use a fast pointer (2 steps)
- If they meet → cycle exists
- If fast reaches null → no cycle

👉 This is the **most efficient and most expected** answer in interviews.

---

## 🟦 2. Using HashSet (Extra Memory)

✔ Time: **O(n)**
✔ Space: **O(n)**

**Idea:**

- Traverse list and store visited nodes in a HashSet
- If a node repeats, cycle exists
- If we reach null, no cycle

### Code:

java Copy code

```java
public boolean hasCycle(ListNode head) {
    Set<ListNode> set = new HashSet<>();

    while (head != null) {
        if (set.contains(head)) return true;
        set.add(head);
        head = head.next;
    }
```

```
        return false;
    }
```

**Why not preferred?**

Uses extra memory.

---

## 🟩 3. Modify Node Values / Marking Nodes (Not recommended)

❌ Not used in interviews
❌ Changes linked list structure
❌ Works only in some situations

**Idea:**

- Mark nodes as visited (e.g., by setting a special value or flag)
- If you hit a visited node again → cycle

But interviewers do not like this approach.

---

## ⭐ Summary

| Approach | Time | Space | Good for Interview? |
|---|---|---|---|
| **Floyd's Tortoise & Hare** | O(n) | O(1) | ⭐⭐⭐⭐⭐ Best |
| **HashSet** | O(n) | O(n) | ⭐⭐⭐ OK |
| **Marking nodes** | O(n) | O(1) | ⭐ Not recommended |

---

## 🗣️ Interview Dialogue (Short)

**Interviewer:**

"How many ways can you detect a cycle in a linked list?"

**You:**

"There are mainly three approaches:

1. Floyd's slow and fast pointer method — best and O(1) space.

2. HashSet to track visited nodes — simple but uses extra memory.
3. Marking nodes — not preferred because it modifies the list."